Ahora tu proyecto solo funciona en **tu computadora** (`localhost`). Al terminar este manual, cualquier persona del mundo podrá abrirlo desde un link como:

```
https://mi-proyecto.onrender.com
```

Para eso necesitamos **dos lugares gratis en internet**:

Pieza	¿Para qué sirve?	Dónde la pondremos (gratis)
La aplicación (Python, Flask, HTML, CSS)	Es la "cara" y el "cerebro" de tu página	**Render**
La base de datos (MySQL)	Es la "bodega" donde se guardan los datos	**Aiven**

Piensa en un restaurante: Render es la cocina y el comedor (donde atiendes a la gente) y Aiven es la bodega (donde guardas los ingredientes). Se comunican por internet.

> **Aviso importante (septiembre 2026):** antes mucha gente usaba PythonAnywhere para esto, pero las cuentas gratis **nuevas** (creadas desde enero de 2026) ya **no incluyen MySQL**. Por eso en este manual usamos Render + Aiven. Los planes gratis cambian con el tiempo, así que si algo no coincide con lo que ves en pantalla, revisa la documentación oficial de cada sitio.

---

2. Lo que necesitas antes de empezar

1. **Python 3** instalado (escribe `python --version` en la terminal para comprobarlo).
2. **Git** instalado (`git --version`).
3. Una cuenta gratis en **GitHub** → https://github.com
4. Una cuenta gratis en **Render** → https://render.com
5. Una cuenta gratis en **Aiven** → https://aiven.io
6. Un editor de código (por ejemplo VS Code).

---

3. Cómo debe estar organizado tu proyecto

Usaremos un proyecto de ejemplo muy sencillo: **un muro de notas** (escribes una nota, se guarda en MySQL y aparece en la lista). Si ya tienes tu propio proyecto, solo adapta tus archivos a esta estructura.

```
mi-proyecto/
│
├── app.py              ← el programa principal de Flask
├── requirements.txt    ← lista de librerías que necesita tu proyecto
├── ca.pem              ← certificado de seguridad de Aiven (lo descargas después)
├── .gitignore          ← archivos que NO se deben subir
│
├── templates/          ← aquí van los HTML
│   └── index.html
│
└── static/             ← aquí van CSS, imágenes y JavaScript
    └── estilos.css
```

Regla de oro de Flask: los HTML van en `templates/` y el CSS/imágenes en `static/`. Con otro nombre de carpeta, Flask no los encuentra.

---

4. El código de ejemplo

4.1 `app.py`

```
import os
import pymysql
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))


def conectar():
    """Abre una conexión con la base de datos MySQL."""
    ruta_certificado = os.path.join(BASE_DIR, "ca.pem")

    # Si existe el certificado (en internet), usamos conexión segura.
    # Si no existe (en tu computadora), nos conectamos normal.
    ssl_config = {"ca": ruta_certificado} if os.path.exists(ruta_certificado) else None

    return pymysql.connect(
        host=os.environ.get("DB_HOST", "localhost"),
        port=int(os.environ.get("DB_PORT", 3306)),
        user=os.environ.get("DB_USER", "root"),
        password=os.environ.get("DB_PASSWORD", ""),
        database=os.environ.get("DB_NAME", "mi_base"),
        ssl=ssl_config,
        cursorclass=pymysql.cursors.DictCursor,
    )


def crear_tabla():
    """Crea la tabla 'notas' si todavía no existe."""
```

```
    try:
        conexion = conectar()
        with conexion.cursor() as cursor:
            cursor.execute(
                """
                CREATE TABLE IF NOT EXISTS notas (
                    id INT AUTO_INCREMENT PRIMARY KEY,
                    texto VARCHAR(255) NOT NULL,
                    creada TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                )
                """
            )
        conexion.commit()
        conexion.close()
    except Exception as error:
        print("No se pudo crear la tabla:", error)


crear_tabla()


@app.route("/")
def inicio():
    conexion = conectar()
    with conexion.cursor() as cursor:
        cursor.execute("SELECT id, texto, creada FROM notas ORDER BY id DESC")
        notas = cursor.fetchall()
    conexion.close()
    return render_template("index.html", notas=notas)


@app.route("/agregar", methods=["POST"])
def agregar():
    texto = request.form.get("texto", "").strip()
    if texto:
        conexion = conectar()
        with conexion.cursor() as cursor:
            # Los %s protegen tu base de datos contra ataques (inyección SQL)
            cursor.execute("INSERT INTO notas (texto) VALUES (%s)", (texto,))
        conexion.commit()
        conexion.close()
    return redirect(url_for("inicio"))


if __name__ == "__main__":
    app.run(debug=True)
```

¿Qué hace cada parte?

- `os.environ.get("DB_HOST", "localhost")` lee un dato desde una **variable de entorno**. Si no existe, usa el valor por defecto (`localhost`). Así **no escribes contraseñas en el código**. Esto es clave.
- `conectar()` abre la comunicación con MySQL.
- `crear_tabla()` crea la tabla sola la primera vez, así no tienes que entrar a ningún panel a crearla.

- `@app.route("/")` define la página principal. `@app.route("/agregar")` recibe el formulario.
- `debug=True` solo se usa en tu computadora. En internet lo manejará `gunicorn` (más abajo).

4.2 `templates/index.html`

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Muro de notas</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='estilos.css') }}">
</head>
<body>
    <main class="contenedor">
        <h1>📝 Muro de notas</h1>

        <form action="{{ url_for('agregar') }}" method="POST">
            <input type="text" name="texto" placeholder="Escribe una nota..." maxlength="255" requ
            <button type="submit">Guardar</button>
        </form>

        <ul class="lista">
            {% for nota in notas %}
                <li>{{ nota.texto }}</li>
            {% else %}
                <li class="vacio">Todavía no hay notas.</li>
            {% endfor %}
        </ul>
    </main>
</body>
</html>
```

Dos detalles importantes:

- `{{ url_for('static', filename='estilos.css') }}` es la forma correcta de enlazar tu CSS en Flask. Funciona igual en tu PC y en internet.
- `{% for ... %}` es un ciclo de Jinja (el sistema de plantillas de Flask) que repite un `<li>` por cada nota.

4.3 `static/estilos.css`

```
* {
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background: #f1f5f9;
    margin: 0;
    padding: 20px;
```

```
}

.contenedor {
    max-width: 500px;
    margin: 40px auto;
    background: white;
    padding: 24px;
    border-radius: 12px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

form {
    display: flex;
    gap: 8px;
}

input {
    flex: 1;
    padding: 10px;
    border: 1px solid #cbd5e1;
    border-radius: 8px;
}

button {
    padding: 10px 16px;
    background: #2563eb;
    color: white;
    border: none;
    border-radius: 8px;
    cursor: pointer;
}

button:hover {
    background: #1d4ed8;
}

.lista {
    list-style: none;
    padding: 0;
    margin-top: 20px;
}

.lista li {
    padding: 10px;
    border-bottom: 1px solid #e2e8f0;
}

.vacio {
    color: #94a3b8;
}
```

4.4 `requirements.txt`

Este archivo le dice al servidor qué librerías instalar:

```
Flask
PyMySQL
cryptography
gunicorn
```

- **Flask**: el framework.
- **PyMySQL**: permite que Python hable con MySQL.
- **cryptography**: PyMySQL la necesita para conexiones seguras.
- **gunicorn**: el servidor que ejecuta tu app en internet (el servidor de pruebas de Flask no sirve para eso).

4.5 `.gitignore`

```
__pycache__/
*.pyc
venv/
.venv/
.env
```

Esto evita subir carpetas inútiles o archivos con contraseñas.

---

5. Prueba primero en tu computadora

Siempre prueba en local antes de subir, así sabes que el error (si lo hay) no es de internet.

```
# 1. Crea un entorno virtual
python -m venv venv

# 2. Actívalo
#   Windows:
venv\Scripts\activate
#   Mac / Linux:
source venv/bin/activate

# 3. Instala las librerías
pip install -r requirements.txt

# 4. Ejecuta
python app.py
```

Abre http://127.0.0.1:5000. Si tienes MySQL instalado localmente, crea una base llamada `mi_base` y configura tus variables. Si **no** tienes MySQL local, puedes saltarte esta prueba y probar directamente con la base de Aiven (paso 7).

> **Si ya tenías un proyecto propio con la contraseña escrita en el código** (por ejemplo `password="1234"`), cámbialo ahora por `os.environ.get(...)` como en el ejemplo. Subir contraseñas a GitHub es uno de los errores más peligrosos.

---

6. Sube el código a GitHub

Render toma tu código desde GitHub. Piensa en GitHub como una "nube" para guardar tu proyecto.

1. Entra a GitHub → botón **New repository** → ponle nombre (ej. `mi-proyecto`) → **Create repository**. Déjalo **vacío** (sin README).
2. En la terminal, dentro de la carpeta de tu proyecto:

```
git init
git add .
git commit -m "Primera versión"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/mi-proyecto.git
git push -u origin main
```

(Cambia `TU_USUARIO` y el nombre del repositorio por los tuyos.)

3. Recarga la página de GitHub: deberías ver tus archivos.

> Cuando GitHub te pida contraseña en la terminal, no uses la de tu cuenta: usa un **Personal Access Token** (GitHub → Settings → Developer settings → Personal access tokens) o inicia sesión con GitHub Desktop / VS Code.

---

7. Crea tu base de datos MySQL gratis en Aiven

1. Entra a https://aiven.io y crea tu cuenta (el plan gratis no pide tarjeta).
2. Pulsa **Create service** y elige **MySQL**.
3. Elige el plan **Free**. Elige una región (cualquiera cercana a ti; la región de Render, abajo, conviene que sea parecida).
4. Ponle un nombre al servicio, por ejemplo `mi-mysql`, y créalo.
5. Espera unos minutos hasta que el estado diga **Running**.

7.1 Copia tus datos de conexión

En la página del servicio verás una sección de conexión con estos datos. Anótalos:

Dato de Aiven	Variable que usaremos
Host	`DB_HOST`
Port	`DB_PORT`
User (normalmente `avnadmin`)	`DB_USER`
Password	`DB_PASSWORD`
Database name (normalmente `defaultdb`)	`DB_NAME`

7.2 Descarga el certificado (CA certificate)

Aiven exige conexión segura. En la misma página hay un enlace **CA certificate** para descargar un archivo.

1. Descárgalo.
2. Renómbralo como `ca.pem`.
3. Cópialo a la carpeta principal de tu proyecto (junto a `app.py`).
4. Súbelo a GitHub:

```
git add ca.pem
git commit -m "Agrego certificado de Aiven"
git push
```

> El certificado CA es público, no es una contraseña, así que es seguro subirlo. **La contraseña NO se sube nunca.**

7.3 Detalle importante: IP permitidas

Por defecto Aiven acepta conexiones desde cualquier IP, que es lo que necesitamos porque Render usa IPs que cambian. Si en algún momento ves un error de "acceso denegado por IP", revisa en Aiven la sección de **Allowed IP addresses** y asegúrate de que esté abierto (`0.0.0.0/0`).

---

8. Despliega la aplicación en Render

1. Entra a https://render.com y regístrate (puedes usar tu cuenta de GitHub, es lo más fácil).
2. Pulsa **New +** → **Web Service**.
3. Conecta tu cuenta de GitHub y elige tu repositorio `mi-proyecto`.
4. Rellena el formulario así:

Campo	Qué poner
Name	`mi-proyecto` (este nombre será parte de tu link)
Region	La más cercana a tu base de datos
Branch	`main`
Runtime / Language	`Python 3`
Build Command	`pip install -r requirements.txt`
Start Command	`gunicorn app:app`
Instance Type	**Free**

¿Qué significa `gunicorn app:app`? Es: "ejecuta, del archivo `app.py`, la variable llamada `app`". Si tu archivo principal se llama `main.py` y tu variable `servidor`, sería `gunicorn main:servidor`.

8.1 Agrega las variables de entorno (¡paso clave!)

Antes de crear el servicio, baja hasta **Environment Variables** → **Add Environment Variable** y agrega una por una con los datos que copiaste de Aiven:

```
DB_HOST      = (el host de Aiven)
DB_PORT      = (el puerto de Aiven)
DB_USER      = avnadmin
DB_PASSWORD  = (tu contraseña de Aiven)
DB_NAME      = defaultdb
```

Aquí es donde viven tus contraseñas: en Render, **no en tu código**.

8.2 Lanza el despliegue

Pulsa **Create Web Service**. Render va a:

1. Descargar tu código de GitHub.
2. Instalar las librerías (Build).
3. Arrancar tu app (Start).

Verás un registro (logs) en pantalla. Cuando aparezca algo como **"Your service is live"**, tu link (arriba a la izquierda) ya funciona. Ábrelo, escribe una nota y guárdala. 🎉

---

9. ¿Cómo actualizo mi página después?

Cada vez que cambies algo (HTML, CSS, Python):

```
git add .
git commit -m "Describe tu cambio"
git push
```

Render detecta el cambio en GitHub y **vuelve a desplegar solo**. No tienes que hacer nada más.

---

10. Lo que debes saber de los planes gratis

- **Render (gratis) "duerme" la app** cuando nadie la visita durante un rato. La siguiente persona que entre puede esperar unos 30–60 segundos mientras despierta. Es normal, no es un error.
- **Aiven (gratis)** puede apagar tu servicio si no se usa durante mucho tiempo (te avisan antes y puedes volver a encenderlo desde el panel). Además, el almacenamiento es limitado y solo puedes tener un servicio MySQL gratis por cuenta.
- Esto es ideal para **tareas escolares, portafolio y pruebas**. Para un negocio real conviene pagar un plan.
- Los planes gratis pueden cambiar. Antes de empezar, confirma las condiciones actuales en https://render.com/pricing y https://aiven.io/pricing/mysql.

---

11. Errores comunes y cómo arreglarlos

Lo que ves	Causa probable	Solución
`ModuleNotFoundError: No module named 'flask'` (o `pymysql`)	Falta la librería en `requirements.txt`	Agrégala al archivo, haz `git push`
`Application failed to respond / Port scan timeout`	El Start Command es incorrecto	Usa `gunicorn app:app` (revisa el nombre de tu archivo)
`No module named 'app'`	Tu archivo principal no se llama `app.py`	Cambia el comando a `gunicorn nombre_archivo:app`
`Can't connect to MySQL server`	Variables de entorno mal escritas o servicio de Aiven apagado	Revisa `DB_HOST` y `DB_PORT`, y que Aiven esté en **Running**
`Access denied for user`	Contraseña o usuario incorrectos	Copia de nuevo los datos de Aiven, sin espacios extra
Error de SSL / certificado	Falta `ca.pem` en el repositorio	Verifica que `ca.pem` esté subido a GitHub y junto a `app.py`
La página se ve **sin estilos**	Ruta del CSS mal escrita	Usa `{{ url_for('static', filename='estilos.css') }}`
`TemplateNotFound: index.html`	La carpeta no se llama exactamente `templates`	Renómbrala (en minúsculas, plural)
En Windows funcionaba y en Render no	Mayúsculas/minúsculas en nombres de archivo	Linux distingue `Estilos.css` de `estilos.css`; usa exactamente el mismo nombre
No sé qué falla	—	En Render, abre la pestaña **Logs**: ahí aparece el error en rojo, léelo de abajo hacia arriba

---

12. Buenas prácticas de seguridad (muy importante)

1. **Nunca** escribas contraseñas dentro del código. Usa variables de entorno.
2. **Nunca** subas archivos `.env` a GitHub (por eso está en `.gitignore`).
3. Usa siempre `%s` en las consultas SQL, como en el ejemplo. **Nunca** armes consultas pegando texto del usuario con f-strings.
4. Si una contraseña se te escapó a GitHub, **cámbiala de inmediato** en Aiven; borrar el archivo no basta porque queda en el historial.
5. En producción no uses `debug=True`. En este manual, gunicorn ejecuta la app y el modo debug solo se

activa cuando corres `python app.py` en tu PC.

---

13. Si usas tu propio proyecto en vez del ejemplo

Revisa esta lista antes de subir:

- ⬛ El archivo principal define `app = Flask(__name__)`.
- ⬛ Los HTML están en `templates/` y el CSS en `static/`.
- ⬛ El `requirements.txt` incluye **todas** las librerías que importas. Para generarlo automáticamente: `pip freeze > requirements.txt` (con tu entorno virtual activado).
- ⬛ Las credenciales de la base de datos vienen de `os.environ.get(...)`.
- ⬛ Si tu base ya tiene tablas y datos en tu computadora, expórtala con `mysqldump -u root -p mi_base > respaldo.sql` e impórtala a Aiven con un cliente como MySQL Workbench, DBeaver o la línea de comandos usando los datos de conexión de Aiven (con el certificado `ca.pem`).
- ⬛ No hay rutas absolutas como `C:\Users\...` en el código.
- ⬛ Hiciste `git push` y el repositorio en GitHub tiene todos los archivos.

---

14. Resumen en 8 pasos

1. Organiza el proyecto: `app.py`, `templates/`, `static/`, `requirements.txt`.
2. Cambia las credenciales del código por variables de entorno.
3. Pruébalo en tu computadora.
4. Súbelo a **GitHub**.
5. Crea tu MySQL gratis en **Aiven** y descarga `ca.pem`.
6. Crea un **Web Service** en **Render** conectado a tu repositorio.
7. Agrega las variables `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`.
8. Abre tu link y listo. Para actualizar, solo haz `git push`.

---

15. Alternativa sin MySQL (por si quieres lo más simple)

Si tu proyecto es solo de práctica y puedes usar **SQLite** en lugar de MySQL, **PythonAnywhere** (https://www.pythonanywhere.com) tiene un plan gratis donde subes tus archivos desde el navegador y obtienes un link `tuusuario.pythonanywhere.com`. Es más fácil de configurar, pero el plan gratis nuevo no incluye MySQL y limita las conexiones salientes, por eso para este manual no es la mejor opción.

---

*¡Felicidades! Ya sabes publicar un proyecto real en internet. Si te sale algún error, copia el mensaje exacto de los **Logs** de Render y búscalo: casi siempre tiene una solución sencilla.*